

EXAMEN DE LA DEUXIEME SESSION

NOM (en majuscules) :	Date: 21.08.2023
Prénom :	Bachelier en Informatique, niveau 1
Local : 124-125-127-128	
UE : Principes algorithmiques et programmation	

Epreuve: **Langage de programmation C#**Professeur : **François KOEUNE**

Matériel autorisé :

Temps :

Consignes:

CRITERES DE REUSSITE = 10/20	A	NA
Pour atteindre le seuil de réussite, <i>l'étudiant sera capable de</i>		
<i>En disposant d'une structure informatique matérielle et logicielle opérationnelle et d'une documentation appropriée, face à un problème mettant en jeu des algorithmes de base, dans le respect du temps imparti</i>		
en C# : développer des programmes en respectant les spécificités du langage choisi - Pouvoir lire et comprendre des fragments de code C# simples (boucle, courte fonction, ...) écrits par un tiers - Pouvoir créer/sauvegarder/exécuter/transmettre un projet C# sous Visual Studio - Pouvoir écrire des programmes C# simples (boucle, courte fonction, utilisation de listes/tableaux,...)		
en C# : mettre en oeuvre des procédures de test - Pouvoir corriger des erreurs majeures de code C# (p.ex. mauvais paramètres lors d'un appel de fonction) sans utiliser d'outil de débogage - Pouvoir utiliser les outils de débogage pour corriger un programme C# - Pouvoir vérifier la correction d'une fonction C# par une série de tests avec différentes valeurs de paramètres		

CRITERES DU DEGRE DE MAITRISE :
Qualité et pertinence de la démarche algorithmique, Rigueur et du respect des spécificités du langage, Degré de précision et clarté dans l'emploi des termes techniques.

COTE GLOBALE :

Points forts	+++	++	+	Points faibles	-	--	---

L'objectif de la première partie de l'examen est de programmer un mini-jeu de démineur. Chacune des questions ci-dessous vise à écrire un élément contribuant à la solution. Quand c'est possible, vous devez utiliser les fonctions écrites pour les réponses précédentes.

Si vous ne pouvez pas répondre à une des questions, vous pouvez quand même indiquer la signature qu'aurait la fonction correspondante, par exemple

```
int Tartempion(string Nom, char Initiale)
```

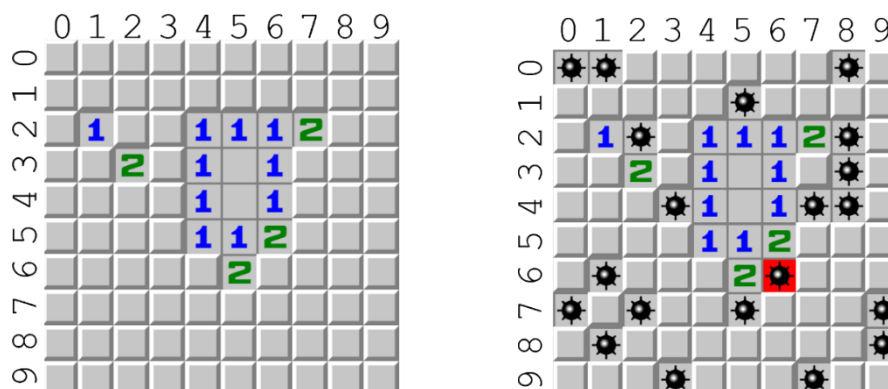
et utiliser cette fonction pour la suite de l'examen : ceci me permettra de valider vos autres réponses.

Principe du démineur

Des mines sont cachées sur un plateau de jeu. Le joueur doit essayer de les trouver, sans jamais tomber sur une. A chaque tour de jeu, le joueur choisit une case :

- Si la case contient une mine, le joueur a perdu.
- Sinon, le jeu affiche le nombre de mines se trouvant autour de cette case.

La figure de gauche montre un exemple de partie en cours, et celle de droite (que le joueur ne voit pas) montre où sont cachées les mines.



Dans cette partie, le joueur a déjà choisi la case (2, 1) (3^{ème} ligne, 2^{ème} colonne) et a appris qu'il y avait une mine autour de cette case. Il a aussi appris qu'il y avait 2 mines autour de la case (3, 2).

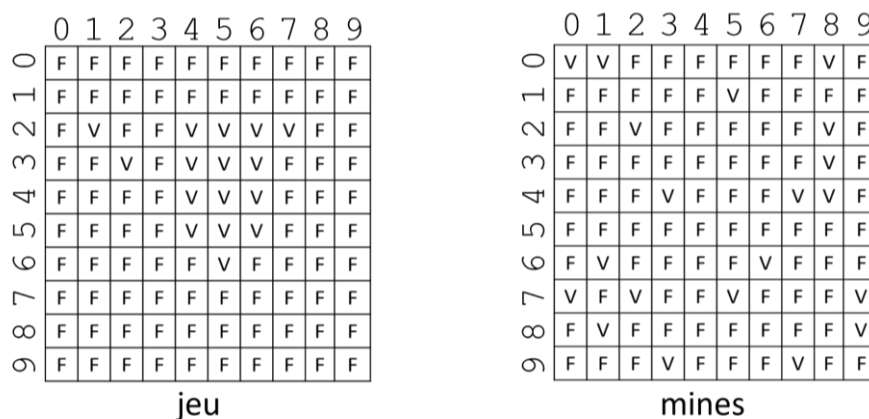
Implémentation du jeu

Nous considérerons un jeu de 10 lignes sur 10 colonnes.

Nous aurons besoin de 2 tableaux à deux dimensions pour représenter le jeu :

- Un tableau de booléens – nommé mines – indiquera où se trouvent les mines.
- Un tableau de booléens – nommé jeu – indiquera quelles cases le joueur a déjà consultées. Le nombre de mines sera recalculé chaque fois que nécessaire.

Les figures ci-dessous montrent donc la représentation interne correspondant aux figures précédentes.



Projet de départ

Récupérez le projet “Demineur”, disponible dans le dossier Examen sur Moodle et décompressez-le localement. A la fin de l’examen, créez une archive compressée de votre projet et chargez-la en réponse à la première partie.

Ce projet, que vous devrez compléter en y ajoutant les réponses aux questions ci-dessous, contient une fonction qui vous sera utile. Il n’est pas nécessaire de comprendre *comment est implémentée* cette fonction, ni de la modifier, il vous suffit de comprendre son *interface* :

- La fonction ChargerTableau() renvoie le tableau mines correspondant aux figures ci-dessus (figures de droite).

Question 1.

Ecrire une fonction CoordonneeValide() qui :

- Reçoit un entier en paramètre.
- Renvoie, sous la forme la plus appropriée, une information indiquant si ce nombre est un numéro de ligne ou de colonne valide, c’est-à-dire compris entre 0 et 9 (inclus).

Question 2.

Ecrire une fonction AccederTableau(). Cette fonction :

- Reçoit 3 paramètres : un tableau de booléens, un numéro de ligne et un numéro de colonne.
- Si le numéro de ligne et le numéro de colonne sont tous les deux valides (entre 0 et 9), la fonction renvoie la valeur de la case correspondante du tableau.
- Sinon, la fonction renvoie false, sans essayer de lire le tableau.

Exemples :

- Avec le tableau de l’exemple ci-dessus, AccederTableau(mines, 0, 1) doit renvoyer true, puisque la case située en 1^{ère} ligne et 2^{ème} colonne vaut vrai.
- Même si les numéros de ligne et colonne sont impossibles, AccederTableau(mines, -1, 15) ou AccederTableau(mines, 1000, 154) ne vont pas déclencher d’erreur : elles vont simplement renvoyer false.

La fonction AccederTableau() va grandement nous simplifier la tâche, car elle nous permettra de lire n’importe quelle case d’un tableau sans devoir nous inquiéter de faire planter le programme si les coordonnées sont invalides. Ceci sera utile pour compter les bombes autour d’une case donnée.

Question 3.

Ecrire une fonction CompterMines(). Cette fonction :

- Reçoit 3 paramètres : un tableau de booléens, un numéro de ligne et un numéro de colonne.
- Compte les mines situées autour de la case visée (y compris en diagonale), et renvoie ce nombre de mines.

Indice : grâce à la fonction AccederTableau(), il n’est pas nécessaire de s’inquiéter que la case visée soit au bord du tableau : tout se passera comme si les cases en dehors du tableau ne contenaient pas de mines.

Question 4.

Ecrire une fonction `Afficher()` qui :

- Reçoit en paramètres deux tableaux de booléens `jeu` et `mines` représentant un jeu de démineur.
- Parcourt le tableau `jeu`. Pour chaque case,
 - Si la case du tableau `jeu` vaut vrai (le joueur a donc déjà regardé cette case), la fonction affiche le nombre de mines présentes autour de cette case.
 - Sinon, la fonction affiche un espace " ".

Question 5.

Ecrire une fonction `LireNombre()` qui :

- Reçoit en paramètre une chaîne de caractères.
- Affiche la chaîne de caractères (qui correspond donc à la question posée à l'utilisateur).
- Lit un entier que l'utilisateur introduit au clavier.
- Si la réponse n'est pas valide (pas comprise entre 0 et 9), la fonction répète la demande jusqu'à obtenir une valeur correcte.
- Enfin, la fonction renvoie l'entier introduit.

Question 6.

Ecrire une fonction `LireCoup()`. Cette fonction :

- Demande à l'utilisateur d'entrer un numéro de ligne, entre 0 et 9, et répète la demande jusqu'à obtenir une valeur correcte.
- Demande à l'utilisateur d'entrer un numéro de colonne, entre 0 et 9, et répète la demande jusqu'à obtenir une valeur correcte.
- Renvoie ces 2 valeurs sous la forme de paramètres out.

Question 7.

Ecrire une fonction `JouerCoup()`.

- Cette fonction reçoit 4 paramètres : deux tableaux de booléens `jeu` et `mines` correspondant à un jeu de démineur, un numéro de ligne et un numéro de colonne (vous pouvez supposer que toutes ces valeurs sont correctes).
- Joue le coup correspondant :
 - Si la case visée correspond à une mine, la fonction affiche « vous avez perdu » et renvoie false.
 - Sinon elle met à jour le tableau `jeu` en indiquant que cette case a été jouée, affiche l'état actuel du jeu (en montrant le nombre de mines autour de chaque case déjà jouée), et renvoie true.

Question 8.

Ecrire une fonction `InitialiserTableau`, qui ne prend pas de paramètres et renvoie un tableau de booléens de taille 10x10 dont toutes les cases valent false.

Question 9.

Ecrire une fonction `JeuDemineur()`. Cette fonction :

- Reçoit en paramètre un tableau de booléens `mines` contenant un certain nombre de mines.
- Crée le tableau `jeu` correspondant, indiquant qu'aucune case n'a été jouée.
- Démarre le jeu, et s'arrête soit après 5 coups, soit dès que le joueur a perdu. Les coups sont joués comme indiqué à la question 7.

Question 10.

Complétez votre programme pour qu'au lancement il adopte le comportement suivant :

- Il charge la grille de test au moyen de la fonction `ChargerTableau()`.
- Il lance la partie au moyen de la fonction `JeuDemineur()`.

Pour la dernière question, qui n'est pas liée aux précédentes, récupérez le projet "ExtraireVoyelles", disponible dans le dossier Examen sur Moodle et décompressez-le localement. A la fin de l'examen, créez une archive compressée de votre projet et chargez-la en réponse à la deuxième partie.

Question 11.

Le programme `ExtraireVoyelles` est censé demander à l'utilisateur d'entrer un mot, puis afficher toutes les voyelles présentes dans ce mot (par exemple, si l'utilisateur tape « `ACCELERER` », le programme doit afficher « `AEEE` »).

Ce programme comporte cependant plusieurs erreurs. Corrigez-le en vous aidant éventuellement des outils de debugging.